

Bitácora de Investigación y Diseño Arquitectónico Co-MIL — Parte III

Continuación del Registro de Desarrollo

Adrián Emiliano Beltrán Fernández

Estudiante de Física Biomédica

Facultad de Ciencias, UNAM

Tutor: Dr. José Eduardo Chairez Veloz

Profesor de Asignatura B, Facultad de Ciencias, UNAM

Ciudad de México, 2026

Documento vivo — última actualización: 21 de agosto de 2026

Resumen

Este documento continúa el registro cronológico de decisiones, dudas metodológicas, rectificaciones e hipótesis del proyecto Co-MIL, iniciado en `co_mil_bitacora.pdf` (Parte I — marco teórico y primer registro de desarrollo, marzo-abril de 2026) y continuado en `co_mil_bitacora_parte2.pdf` (Parte II — fase de experimentación y refinamiento, abril de 2026). No reproduce el contenido de esas dos partes; se referencian por nombre cuando es necesario dar contexto. A partir de aquí, la bitácora se actualiza de forma incremental, sesión por sesión, conforme avanzan el desarrollo y la investigación del proyecto.

Índice general

1. Registro de Desarrollo — Continuación	2
1.1. Contexto de reanudación	2
1.2. 21 de agosto de 2026	2
1.3. 22 de agosto de 2026	5
1.4. 23 de agosto de 2026	10
1.5. 27 de agosto de 2026	18

Capítulo 1

Registro de Desarrollo — Continuación

1.1. Contexto de reanudación

Entre el cierre de la Parte II (finales de abril de 2026) y la reanudación de este registro (agosto de 2026) hubo una pausa prolongada de actividad en el proyecto. Se retoma el trabajo con dos objetivos inmediatos: (1) someter el estado real del proyecto a una auditoría crítica externa antes de seguir invirtiendo tiempo de anotación clínica, y (2) preparar un Producto Mínimo Viable (PoC) presentable para la siguiente reunión con el tutor.

1.2. 21 de agosto de 2026

Auditoría crítica externa del proyecto. *Solicitada por el autor con el objetivo explícito de elevar el rigor científico y técnico del proyecto a nivel de tesis de maestría antes de continuar la anotación clínica.*

Se realizó una revisión línea por línea de todo el pipeline (`dataset.py`, `entrenar_comil.py`, `evaluar_comil.py`, `Models/attention_mil.py`, `generador_bolsas.py`, `reprocesador_dataset.py`), del Plan de la Práctica Profesional Supervisada (`Plan_PPS.pdf`), de las bitácoras previas, y un escaneo programático de las 204 bolsas `.pt` generadas hasta la fecha (58 de las 100 imágenes de `Dataset_Experto_100`). Hallazgos principales:

- El componente de *aprendizaje continuo* (rehearsal, regularización o módulos incrementales) — el elemento que le da nombre al proyecto (la “Co” de Co-MIL) — no tiene todavía ninguna línea de implementación. Lo que existe es un clasificador de Aprendizaje Multi-Instancia (MIL) de una sola pasada.
- `evaluar_comil.py` evalúa el modelo sobre la misma carpeta de datos que `entrenar_comil.py` usa para entrenar: no existe partición de entrenamiento/validación/prueba en ningún script. Cualquier métrica producida hasta ahora mide memorización, no generalización.
- El catálogo de clases de tejido está fragmentado en 4 esquemas incompatibles (vectores de etiqueta de longitud 10, 11 y 12, con nombres de clase que no coinciden exactamente entre sí) repartidos de forma no controlada entre las 204 bolsas ya generadas. La causa raíz es

que la herramienta de anotación (`generador_bolsas.py`) permite modificar el catálogo de clases en cualquier momento sin migrar las bolsas ya guardadas.

- La arquitectura base (MobileNetV2 congelado + Atención con Compuertas de K ramas independientes, siguiendo a Ilse et al., ICML 2018) está bien fundamentada y ya fue validada mediante un experimento sanity-check sobre bolsas sintéticas de MNIST (documentado en la Parte I), pero nunca se ha entrenado ni evaluado de forma metodológicamente válida sobre datos clínicos reales.
- El `Plan_PPS.pdf` corresponde a la Práctica Profesional Supervisada, ya calificada y entregada — **no** es el plan de tesis formal, que todavía está por redactarse y tramitarse. Esto da margen real para actualizar bibliografía, alcance y arquitectura antes de comprometer el plan de tesis por escrito.

Diagnóstico cuantitativo de las anotaciones existentes. *Se corrigió una hipótesis de riesgo planteada en la auditoría inicial.*

Un primer diagnóstico programático (usando el vector de etiquetas congelado Y) sugería un posible riesgo de colapso de varianza (“*síndrome de la red perezosa*”, ya anticipado por el autor en la Parte I): que la mayoría de las bolsas tuvieran combinaciones de tejidos casi idénticas. Un segundo diagnóstico, más confiable, usando las etiquetas crudas (`roi_labels`, que no dependen del catálogo vigente al momento de guardar), mostró que la combinación más frecuente de etiquetas por bolsa es, de hecho, un solo tejido a la vez (45 bolsas con únicamente “Tejido de Granulación”, 37 con únicamente “Fibrina (Esfacelo)”, etc.). Esto confirma que el cambio de criterio de anotación adoptado por el autor a mitad del lote — separar tejidos en regiones de interés (ROI) distintas en vez de agruparlos en una sola caja delimitadora — sí mitigó el riesgo identificado, aunque de forma no documentada hasta ahora. Se decide formalizar este cambio de criterio como “protocolo de anotación v2” (frente al “v1” de las primeras imágenes), y adoptarlo de forma consistente en lo que resta del lote.

También se confirmó que la clase “Tendón / Músculo Expuesto” tiene una prevalencia real muy baja (1 de 204 bolsas, 0.5%), lo cual deberá tenerse en cuenta en el cálculo de `pos_weight` y, eventualmente, al decidir si se conserva como clase independiente.

Decisiones tomadas. *Registradas aquí para no reabrir la discusión en sesiones futuras.*

- **Nombre canónico del catálogo de clases:** se fija en 12 clases, con la clase 0 nombrada “*Tejido de Granulación*” (coincide con el catálogo vigente en `custom_classes.json`, y con la mayoría de las bolsas ya guardadas).
- **Arreglo de raíz del catálogo fragmentado:** en vez de migrar destructivamente las 204 bolsas existentes, se rediseñará `dataset.py` para calcular el vector de etiquetas Y *en tiempo de carga*, a partir de `roi_labels` (los nombres de tejido tal como se marcaron, que sí se conservan íntegros en cada bolsa) contra el catálogo canónico vigente — en vez de confiar en el Y congelado al momento de guardar. Esto hace el sistema tolerante a que el catálogo siga evolucionando entre sesiones.

- **Clase de control / tejido sano:** no se crea una clase nueva de “No tejido”; se formaliza el uso de la clase ya existente “Piel Perilesional Sana” para ese propósito (ya se había usado, sin criterio explícito, en 9 de las 204 bolsas).
- **Datasets públicos** (DFUTissue, FUSeg, DFUC 2021): no se usarán para entrenamiento (paradigma de anotación distinto al de Co-MIL, basado en máscaras densas en vez de etiquetas débiles por región), pero sí se evaluará usarlos como *validación externa*, mediante un mapeo explícito de las 12 clases finas de Co-MIL a las 3–4 clases gruesas que comparten esos datasets públicos.
- **Vara de medir de referencia:** se adopta como referencia realista (no como meta a superar) el desempeño reportado en el artículo propio “AI-based Mobile App for Segmentation and Tissue Classification on Diabetic Foot Ulcer” (coautoría del autor), que con supervisión densa completa y un dataset mucho mayor reporta Dice de 78.6 % en Granulación, 51.5 % en Callo y solo 33.3 % en Fibrina.
- **Organización del repositorio:** el material de proceso (notas de investigación, auditorías internas, scripts de diagnóstico) se mantiene separado del código de producción del proyecto en una carpeta de trabajo local excluida de control de versiones, para que el repositorio público quede limpio y enfocado únicamente en el sistema Co-MIL.
- **Reunión con el tutor:** programada para aproximadamente el 28 de agosto de 2026 (una semana a partir de esta entrada). Se define una hoja de ruta de 7 días priorizando un PoC técnico válido (arreglo de catálogo, partición de datos reproducible, evaluación correcta, primer entrenamiento) y un diseño — al menos parcial — del componente de aprendizaje continuo, por encima de terminar de anotar la totalidad del lote de imágenes.

Investigación dirigida de estado del arte — resultados. *Búsqueda dirigida (no revisión sistemática) sobre cinco preguntas concretas y citables; el detalle completo con enlaces queda en las notas de investigación internas del autor.*

- **Novedad central:** no se encontró trabajo alguno que combine aprendizaje continuo con MIL sobre fotografías macroscópicas de UPD o heridas crónicas en general. La afirmación de novedad es defendible, con la salvedad metodológica de presentarla como “no encontrado en literatura indexada en inglés hasta la fecha de la búsqueda” y no como ausencia absoluta.
- **Citas de Co-MIL en WSI (histopatología):** se confirmaron como reales y correctas — Li, Cui, Li & Chan, “*Advancing Multiple Instance Learning with Continual Learning for Whole Slide Imaging*”, CVPR 2025 (arXiv:2505.10649); Lee, Jeong, Han, Lee & Chun, “*Continual Multiple Instance Learning with Enhanced Localization for Histopathological Whole Slide Image Analysis*” (CoMEL), ICCV 2025 (arXiv:2507.02395). Se identificó un antecedente adicional no citado hasta ahora: Meseguer, del Amor & Naranjo, “*MICIL: Multiple-Instance Class-Incremental Learning for skin cancer whole slide images*”, Artificial Intelligence in Medicine (2024).

- **Taxonomía de 12 clases:** la afirmación de que ningún trabajo tiene tantas clases se sostiene, pero debe matizarse — el máximo público encontrado es de 6 clases (Kabir et al., *WoundTissueSeg*, arXiv:2502.10652, 2025, y su extensión *WoundFormer*, arXiv:2605.19868, 2026), no “3–4 clases” como se pensaba inicialmente.
- **Atención simple (Ilse et al. 2018) frente a agregadores tipo transformer:** no se encontró una comparación empírica directa en datasets pequeños citable textualmente; sí hay apoyo indirecto fuerte en la proliferación de arquitecturas MIL ligeras diseñadas para mitigar justo ese problema (DTFD-MIL, CVPR 2022; FALFormer, 2024; LiteMIL, 2025). Debe presentarse como inferencia razonada, no como cita directa.
- **Implementación de búfer de repetición:** ruta recomendada es la librería Avalanche (ContinualAI), con `ReplayPlugin` ya listo para usar, o una implementación manual mínima con *reservoir sampling* (50 líneas) siguiendo el patrón de `GMvandeVen/continual-learning`.

Con esto se da por cerrada la sesión de trabajo del 21 de agosto de 2026.

1.3. 22 de agosto de 2026

Etapas 0 completada: arreglo de raíz del catálogo de clases. *Día 2 del plan de 7 días.*

Se implementó y verificó de punta a punta la corrección estructural del catálogo de clases fragmentado:

- Se creó `C0-MIL/catalogo_tejidos.py`, módulo compartido entre la herramienta de anotación (`generador_bolsas.py`) y la ingesta de datos (`dataset.py`), que reemplaza y absorbe la lógica de `comil_ia_pipeline.py` (eliminado).
- `dataset.py` ya no lee el vector **Y** congelado de cada bolsa; lo reconstruye en tiempo de carga a partir de las etiquetas crudas (`roi_labels`) contra el catálogo canónico vigente, resolviendo sinónimos y renombrados históricos.
- `generador_bolsas.py` ahora exige la firma del anotador antes de guardar (ya no cae en “Anónimo” en silencio), y registra automáticamente cada renombrado de clase en `renombres_clases.json` para no perder la trazabilidad hacia atrás.
- Se eliminó la copia duplicada de `custom_classes.json` en `APP_generador_bolsas/` (no era la que realmente usaba la aplicación en modo script).

Verificación end-to-end: se cargaron las 204 bolsas reales del lote de la experta clínica con el nuevo `dataset.py` sin ningún error. Las 204 bolsas quedaron con un vector **Y** de longitud 12 de forma consistente (antes: 34 bolsas de longitud 10, 69 de longitud 11, 101 de longitud 12), y cero etiquetas crudas sin reconocer. La prevalencia por clase recalculada confirma los números ya vistos en el diagnóstico del día anterior, con el ajuste esperado en “Tejido de Granulación” (58/204, 28.4 %) al fusionarse correctamente con las bolsas que usaban el sinónimo histórico “Granulación”. Las clases “Hueso Expuesto” (0/204) y “Tendón

/ Músculo Expuesto” (1/204) quedan documentadas como las de menor prevalencia real en el dataset actual.

Queda pendiente, no bloqueante: `reprocesador_dataset.py` (redimensionamiento multi-resolución) todavía no guarda `roi_labels` en las bolsas que reprocesa, por lo que perdería la protección de la vectorización dinámica si se usa antes de corregirlo; no es urgente porque ese script no se ha usado todavía en este dataset.

Próxima entrada: Etapa 2 del roadmap (partición de datos reproducible a nivel imagen, aumento de datos, y reescritura de `evaluar_comil.py` con métricas válidas).

Etapla 2: primera evaluación válida en la historia del proyecto. *Mismo día, continuación de la sesión (22 de agosto de 2026).*

Se implementó el resto de la Etapa 2 y se corrió el pipeline completo de punta a punta por primera vez:

- `CO-MIL/particionar_dataset.py` (nuevo): arma una partición train/val/test reproducible a nivel de *imagen* (nunca a nivel de ROI, para no filtrar información), con semilla fija y un manifiesto versionado (`splits_manifest.json`) que preserva las asignaciones ya hechas cuando el dataset crece. Corrido sobre las 58 imágenes actuales: 40 entrenamiento / 9 validación / 9 prueba.
- `dataset.py` ahora soporta filtrar por partición y aplicar aumento de datos (flip, rotación, color) por parche, activado solo en entrenamiento.
- `evaluar_comil.py` evalúa exclusivamente sobre el split de prueba (nunca el de entrenamiento) y agrega AUC-ROC, sensibilidad y especificidad por tejido.

Al ejecutar el entrenamiento real por primera vez aparecieron dos problemas que ninguna revisión estática anterior pudo detectar, porque el pipeline nunca se había corrido contra datos clínicos: faltaban las dependencias `torchmil` y `seaborn` en el entorno, y la función `torchmil.data.collate_fn` de la versión instalada (1.0.2) espera que cada bolsa se entregue como diccionario, no como tupla — una discrepancia de API frente a lo que asumía el código original. Se corrigió en `dataset.py`, `entrenar_comil.py` y `evaluar_comil.py`.

Resultado: se entrenó el modelo base (30 épocas, backbone congelado, 122 bolsas de entrenamiento con aumento de datos) — la pérdida promedio bajó de 1.23 a 0.84. Evaluado sobre el split de prueba nunca visto (37 bolsas / 9 imágenes): Hamming Loss 0.2545, Subset Accuracy 0.162. Por tejido, AUC-ROC entre 0.61 y 0.90 para las clases con suficiente soporte (Tejido Calloso 0.900, Epitelización 0.894, Tejido de Granulación 0.846, Fibrina 0.710, Tejido Necrótico 0.614); cuatro clases (Exudado, Hueso Expuesto, Tendón/Músculo, Piel Perilesional Sana) no tuvieron ningún caso positivo en las 9 imágenes de prueba y quedan sin definir por ahora. Comparado con el artículo de Italia (supervisión densa completa, más de 1000 imágenes): la clase que ahí resultó más difícil de distinguir (Fibrina, Dice 33 %) es también la más difícil aquí entre las que sí tienen señal (AUC 0.71), y la más fácil allá (Granulación, Dice 78.6 %) mantiene señal fuerte aquí (AUC 0.85) — consistencia direccional razonable entre ambos enfoques, aunque Dice y AUC-ROC no son directamente comparables en magnitud.

Checkpoint guardado en `Pesos_Entrenados/comil_miml_fase1.pth`; matrices de confusión en `Pesos_Entrenados/matrices_confusion_test.png`.

Pendiente, no bloqueante: validación cruzada (el manifiesto ya asigna un *fold* a cada imagen, falta el bucle que lo use).

Corrección de cobertura del split y reentrenamiento. *Continuación de la misma sesión.*

Al revisar por qué algunas clases (Piel Perilesional Sana, Maceración/Eritema) parecían no tener casos positivos, se encontró que sí estaban anotadas (5 y 16 ejemplos respectivamente solo en entrenamiento) — lo que faltaba era que el split de prueba las representara. El algoritmo de partición (`particionar_dataset.py`) tenía dos rondas de bug real:

- Un conteo duplicado que sesgaba la partición lejos de las proporciones pedidas (70/15/15 salía como 52/24/24).
- Una pasada de reparación de cobertura mínima que, cuando la misma imagen era necesaria para cubrir dos clases distintas en splits distintos, dejaba que la segunda corrección sobrescribiera la primera en silencio.

Corregido ambos, con un reporte explícito de cobertura impreso cada vez que se corre el script. Resultado final: 10 de 12 clases con al menos un ejemplo en los tres splits; las dos excepciones (Hueso Expuesto con 0 ejemplos en todo el dataset, Tendón/Músculo con solo 1) son limitaciones reales de datos, no del algoritmo, y quedan documentadas automáticamente.

Se reentrenó el modelo con la partición corregida (36 train / 46 test bolsas). Evaluación sobre el split de prueba:

Cuadro 1.1: Desempeño por tejido, split de prueba (46 bolsas), tras la corrección de cobertura.

Tejido	Positivos	Sensibilidad	Especificidad	AUC-ROC
Exudado	1	1.000	0.444	0.956
Piel Perilesional Sana	2	1.000	0.432	0.898
Tejido de Granulación	13	0.846	0.636	0.890
Tejido Calloso (Hiperqueratosis)	7	1.000	0.744	0.883
Hiperpigmentación	4	0.750	0.833	0.863
Maceración / Eritema	1	1.000	0.578	0.844
Epitelización	6	0.500	0.825	0.796
Fibrina (Esfacelo)	7	0.571	0.744	0.722
Tejido Necrótico (Escara)	5	0.400	0.732	0.698
Hipergranulación	1	0.000	0.667	0.667
Hueso Expuesto	0	N/D	1.000	N/D
Tendón / Músculo Expuesto	0	N/D	1.000	N/D

Hamming Loss = 0.2790, Subset Accuracy = 0.0435 (exige acierto exacto en las 12 clases a la vez, muy exigente con tan pocos datos). Las tres clases que antes salían sin definir ahora muestran AUC alto (0.86–0.96) en cuanto el split las deja evaluar — confirma que la señal ya estaba ahí.

Refinamiento DenseCRF: implementación, depuración y motor intercambiable. *Solicitado explícitamente para mejorar la interpretación clínica de los mapas de atención (ver discusión de granularidad más abajo).*

`pydensecrf` y su alternativa `SimpleCRF` no se pudieron instalar (requieren compilar extensiones en C++/Cython y este equipo no tenía un compilador de Visual Studio configurado). Se implementó `CO-MIL/refinamiento_crf.py`: el mismo formalismo de campo medio ya definido en esta bitácora (ecuaciones 8.1–8.2, Parte I) en NumPy puro. Tres bugs reales antes de que quedara bien, cada uno verificado visualmente contra datos reales, no solo revisando que corriera sin error:

1. Aliasing periódico al reducir la imagen a la resolución de trabajo de un solo salto — corregido con reducción progresiva (a la mitad en cada paso, tipo pirámide de imagen).
2. El radio de la ventana del filtro bilateral era mucho más chico que el sigma espacial pedido, así que la ventana se comportaba como una caja rígida en vez de una campana suave — ahora la función exige $\text{radio} \geq 2,5\sigma$ o truena.
3. La iteración de campo medio sin amortiguar era inestable: más iteraciones producían un patrón fractal en vez de converger — corregido mezclando 50/50 el resultado propuesto con el paso anterior en cada iteración.

El módulo quedó con **motor intercambiable**: si `pydensecrf` está instalado (más rápido, corre a resolución completa, es la implementación que cita la literatura de WSSS), se usa automáticamente; si no, usa el respaldo en NumPy verificado. Ningún otro script necesita saber cuál de los dos está activo. Con Visual Studio ya instalado en el equipo del autor, falta solo el componente “Desktop development with C++” para poder instalar `pydensecrf` real.

La Figura 1.1 resume el estado completo del pipeline a esta fecha, y la Figura 1.2 el detalle de la arquitectura del modelo (`CoMILNetwork`).

Pipeline Co-MIL — estado del proyecto al 22 de agosto de 2026

Recuadro punteado: componente aún no implementado (siguiente etapa del roadmap).

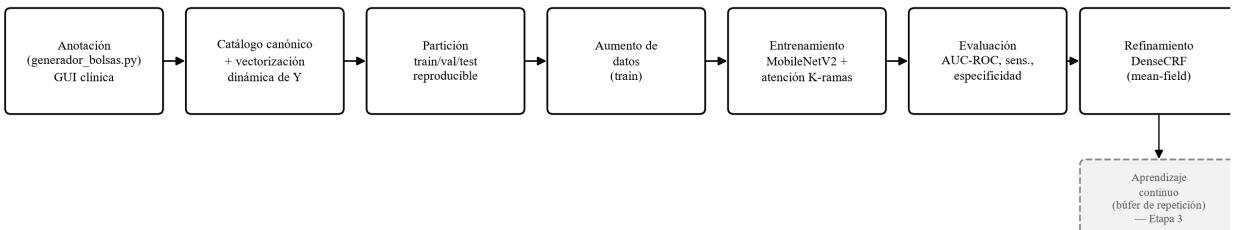


Figura 1.1: Pipeline completo de Co-MIL al 22 de agosto de 2026. El recuadro punteado (aprendizaje continuo) es la única etapa mayor del plan de tesis todavía sin implementar.

Arquitectura CoMILNetwork (Models/attention_mil.py)

K = número de tejidos del catálogo vigente (12 en el dataset actual). Cada tejido tiene su propia rama de atención y su propio clasificador.

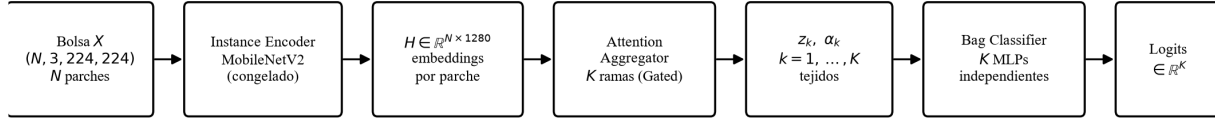


Figura 1.2: Arquitectura CoMILNetwork: MobileNetV2 congelado como extractor de características, seguido de K ramas de atención independientes (una por tejido) y K clasificadores MLP independientes.

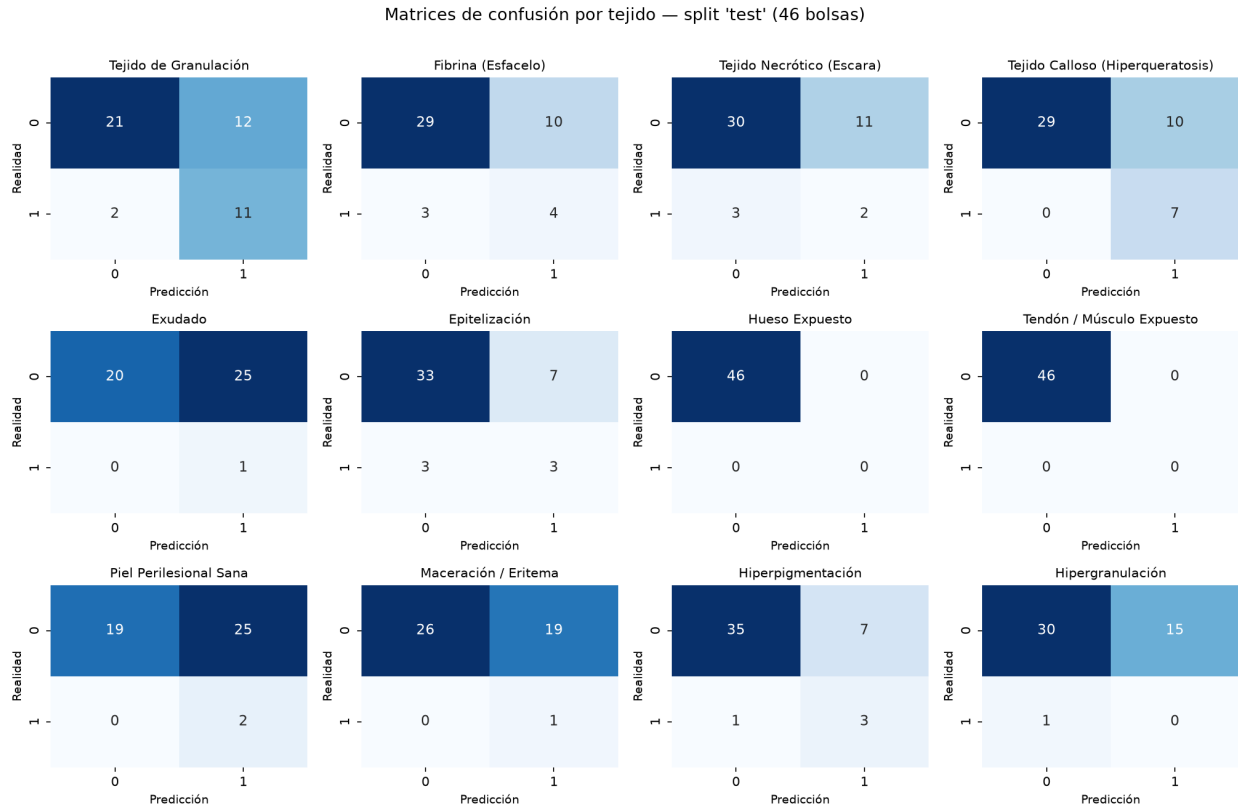


Figura 1.3: Matrices de confusión por tejido sobre el split de prueba (46 bolsas), modelo reentrenado con la partición corregida.

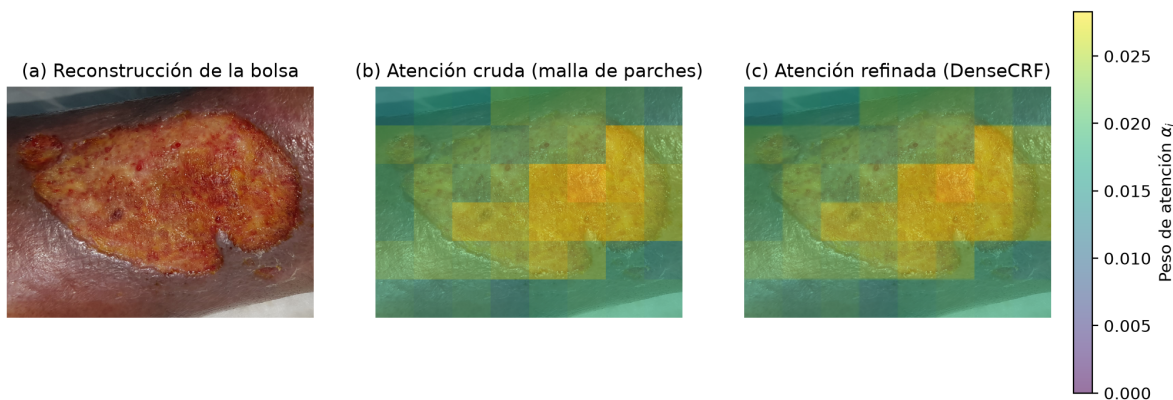


Figura 1.4: Ejemplo real (split de validación, imagen UPD_013, tejido “Fibrina (Esfacelo)”) del efecto del refinamiento DenseCRF: (a) reconstrucción de la bolsa, (b) atención cruda a nivel de parche, (c) atención refinada. El efecto es sutil en este caso porque la atención cruda del modelo todavía está poco concentrada (rango ~ 0.015 – 0.028 sin picos claros) — esperable con solo 30 épocas de entrenamiento y tan pocos datos; se espera un efecto más marcado conforme el modelo converja mejor.

Discusión: granularidad espacial de los mapas de atención. *Misma pregunta que el autor dejó planteada desde el inicio de esta bitácora, retomada explícitamente hoy.*

El heatmap actual tiene, como máximo, un valor por parche de 224×224 px — para una ROI de pocos cientos de píxeles esto deja una malla de 2×2 o 3×3 celdas, insuficiente para interpretación clínica fina de bordes de tejido. Dos rutas, complementarias, ambas ya sustentadas en el diseño existente del pipeline (no requieren rediseño desde cero):

1. **Refinamiento DenseCRF** (implementado hoy, ver arriba): barato, no requiere reprocesar datos ni más cómputo de entrenamiento.
2. **Parches más finos:** `reprocesador_dataset.py` ya sabe re-extraer bolsas a 56 o 112 px en vez de 224 px desde la imagen original en alta resolución, y `dataset.py` ya reescala esos parches chicos a 224 px solo para la entrada del CNN, conservando la malla fina para el mapa de atención. Requiere primero que `reprocesador_dataset.py` propague `roi_labels` (pendiente, ver Etapa 0 del roadmap), y es más caro en cómputo — probablemente necesite GPU/Colab.

Con esto se da por cerrada la sesión de trabajo del 22 de agosto de 2026.

1.4. 23 de agosto de 2026

Seguimiento de experimentos: cada corrida en su propia carpeta. *Motivado por confusión real al revisar resultados: había figuras de dos modelos distintos mezcladas en el mismo lugar sin ninguna forma de distinguirlas.*

Nuevo módulo CO-MIL/`experimentos.py`: cada corrida de `entrenar_comil.py` ahora crea su propia carpeta `Pesos_Entrenados/exp_<fecha>_<hora>/` con un `metadata.json` que registra la resolución de parche (escaneada de las bolsas reales, no asumida), el desglose `train/val/test`, número de clases e hiperparámetros. `evaluar_comil.py` y `visualizar_resultados.py` escriben sus resultados dentro de esa misma carpeta y cada imagen trae en el título la resolución de parche, la malla y el nombre del experimento. El checkpoint del 22 de agosto se migró a `exp_20260822_172138_split-corregido/` sin reentrenar; los archivos sueltos viejos se archivaron aparte con nombres explícitos.

Discusión crítica: qué puede y qué no puede resolver DenseCRF. *El autor observó, revisando el Artifact con el ejemplo de UPD_013, que la malla de atención cruda y la refinada tenían exactamente la misma cuadrícula — solo cambiaban los colores. Se investigó a fondo antes de seguir.*

Se midió el contraste de atención (máximo menos mínimo) en las 47 combinaciones bolsa-clase positivas del split de prueba: la **mediana es de solo 0.04** (en escala 0–1). El caso de mayor contraste en todo el split (`UPD_035__roi_05_bag.pt`, tejido calloso, rango 0.05–0.27) se usó para verificar el mecanismo con una imagen real: el refinamiento sí desplaza ligeramente un par de fronteras hacia bordes reales de la piel (Figura 1.5), pero nunca genera detalle *dentro* de un bloque de 224px, porque el término unario (la atención cruda del modelo) no tiene ninguna información a esa escala — no es un error del código, es una limitación matemática esperable del método: el CRF refina lo que ya existe, no inventa resolución que el parche nunca calculó.

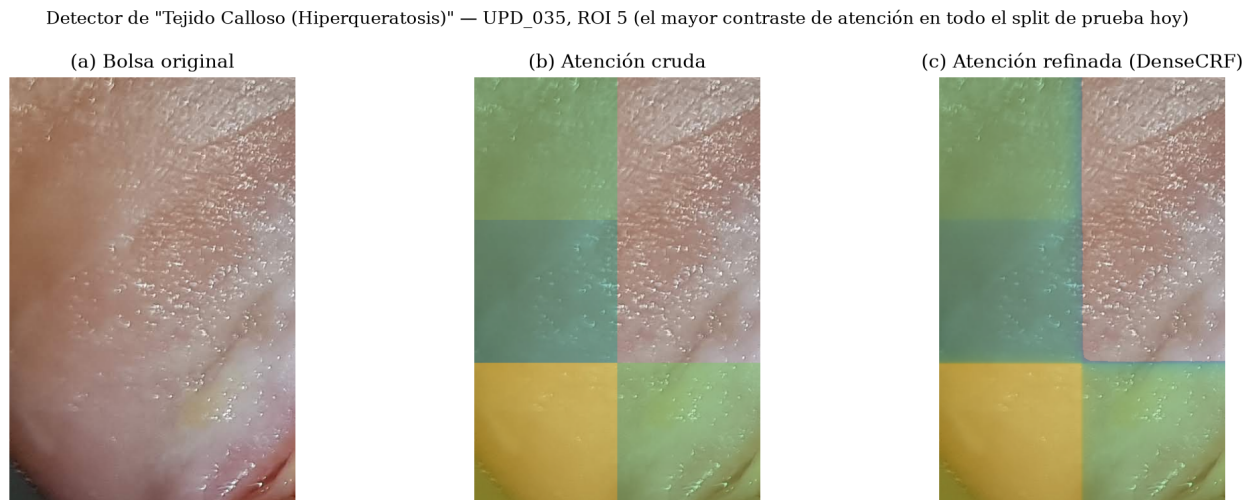


Figura 1.5: El caso de mayor contraste de atención disponible en todo el split de prueba (23 de agosto de 2026). El refinamiento desplaza fronteras hacia bordes reales de la piel, pero no genera detalle dentro de un bloque — confirma que la limitación es de resolución de la señal de entrada, no del algoritmo de refinamiento.

Conclusión: para lograr precisión espacial útil clínicamente (el objetivo explícito es acercarse a algo parecido a segmentación multiclase, sin llegar a serlo), la vía real es reducir el

tamaño de parche, no seguir ajustando el CRF. También se corrigió la presentación visual de los heatmaps (antes `jet` con alta opacidad, que tapaba la úlcera real): ahora usan `viridis`, opacidad baja con máscara por umbral, y normalización por bolsa (necesaria porque con rangos de atención tan chicos, una escala de color absoluta 0–1 se ve plana en casi todos los casos).

Costo computacional de reducir el tamaño de parche. *Pregunta directa del autor: ¿es posible llegar a parches de 20×20 px?*

El número de parches por bolsa escala con $(224/P)^2$ para una ROI del mismo tamaño físico. La bolsa más grande hoy (N=48, malla 6×8 a 224px) se transforma así:

Cuadro 1.2: Costo relativo de reducir el tamaño de parche (bolsa más grande del dataset actual).

Tamaño de parche	Parches (N)	Factor de cómputo	Factibilidad
224 px (actual)	48	1×	ya corre en CPU
112 px	~192	~4×	probable en CPU hoy mismo (1–1.5h)
56 px	~768	~16×	necesita GPU/Colab
20 px	~6,000+	~125×	no viable ni con GPU modesta

Además del costo, hay una duda independiente: un parche de 20px puede no contener suficiente textura para que MobileNetV2 extraiga una característica útil por instancia. **Decisión pendiente:** 112px es la prueba realista para hoy; 56px queda condicionado a tener GPU/Colab disponible; 20px se descarta como meta directa por ahora.

Por qué las métricas de clasificación salen bien con atención casi plana. *Pregunta directa del autor: si el modelo no distingue entre parches, ¿cómo llega a esas métricas?*

Se verificó: **97.8% de las bolsas de prueba (45 de 46) contienen un solo tipo de tejido en todos sus parches.** Como el logit de la bolsa sale de $z = \sum_i \alpha_i h_i$ (promedio ponderado por atención), una atención casi uniforme simplemente significa que z es casi el promedio simple de los parches — y como la bolsa ya es homogénea, ese promedio es una firma limpia y representativa del tejido, fácil de clasificar bien. El AUC alto es real y válido para clasificación; no implica que la atención haya aprendido a localizar nada, porque el propio recorte manual de ROIs pequeñas y casi de un solo tejido (protocolo v2) ya hizo ese trabajo de localización a mano — no queda heterogeneidad dentro de la bolsa para que la atención busque.

Esto conecta con el Manual de Usuario original (9 de agosto de 2026), que pedía dejar 15–25% de piel sana alrededor de la úlcera como referencia visual para el modelo— ese margen generaba heterogeneidad real dentro de la bolsa (tejido + piel sana mezclados) que sí le habría dado a la atención algo genuino que localizar. El protocolo v2 mejoró la limpieza de la señal de clasificación pero, sin buscarlo, redujo también la heterogeneidad que la atención necesita para ser informativa. Los dos problemas (parche grueso y bolsa homogénea) apuntan hacia la misma solución.

Propuesta creativa: MIL denso estilo CAM. *Solicitada explícitamente por el autor ("sé creativo, propón soluciones innovadoras, incluso una metodología nueva"). La tesis aún no*

se presenta formalmente, así que el alcance todavía se puede ajustar — pero debe decidirse pronto.

Propuesta concreta: en vez de teselar la ROI en parches independientes, pasar la ROI completa por la red convolucional **una sola vez**, y usar el mapa de características nativo que la CNN ya produce como malla de instancias, en vez de un mallado de 224px trazado a mano. MobileNetV2 reduce la imagen $\sim 32\times$; una ROI de 896×1152 px daría una malla de $\sim 28\times 36$ — mucho más fina que la malla actual de 6×8 , y más barata en cómputo (una sola pasada de toda la imagen, no N pasadas de parches por separado). El cambio de código es quirúrgico: solo se reemplaza **InstanceEncoder**; **AttentionAggregator** (las K ramas de atención) y **BagClassifier** no cambian, porque ya operan sobre un conjunto genérico de vectores de instancia sin importar cómo se produjeron. No es abandonar MIL: es la línea de MIL basado en *pooling*/CAM (ej. Pinheiro & Collobert, 2015, "From image-level to pixel-level labeling with CNNs"), una variante real y citable, no una invención sin respaldo. Complementaria (más esfuerzo, segunda fase si la malla densa no basta): instancias por superpíxel (SLIC) en vez de malla rígida, con fronteras que siguen el color/textura real de la piel.

Estado: propuesta, aún no diseñada a detalle ni implementada. Plan acordado: hoy se prueba el tamaño de parche de 112px sobre la arquitectura actual; esta propuesta se diseña con calma la siguiente sesión.

Pregunta crítica de despliegue: recorte de ROI en producción. *Igualmente señalada como crucial por el autor.*

Todo el pipeline actual — entrenamiento e inferencia — asume que se le entrega una ROI ya recortada (trazada a mano con `generador_bolsas.py`). No existe ningún paso automático de localización de la úlcera. Esto no es un hallazgo nuevo: la bitácora original ya había planeado un localizador automático basado en U-Net y lo pospuso explícitamente el 6 de abril de 2026 por alcance de MVP. La pregunta del autor es legítima y queda abierta: ¿el usuario final deberá recortar o hacer zoom a la úlcera antes de pasar la foto por el modelo (más simple, replica el flujo actual de anotación), o hay que construir el localizador automático que se pospuso? Dado el tiempo disponible antes de la reunión con el tutor, la recomendación es **no** construir el localizador esta semana, pero sí documentar explícitamente como requisito/limitación del sistema actual que la entrada debe ser una ROI ya recortada — en vez de dejarlo implícito.

Tensión real entre anotación débil y esfuerzo de anotación efectivo. *Reflexión del autor sobre el planteamiento original del tutor.*

La justificación original del tutor para usar MIL fue que la anotación débil (etiqueta a nivel de bolsa, sin máscara) sería más rápida que la anotación densa. En la práctica, conseguir una señal de atención limpia ha llevado a trazar ROIs cada vez más pequeñas y casi de un solo tejido (algunas imágenes ya superan 45 bolsas), acercando el esfuerzo de anotación al de una segmentación casi completa. Esto es una tensión real y reconocida en la literatura de aprendizaje débilmente supervisado (entre más "limpias/finas las bolsas, mejor la localización pero menor el ahorro de anotación respecto a supervisión completa) — no es un error de ejecución del autor. Vale la pena discutirlo explícitamente con el tutor y documentarlo como limitación reconocida en la tesis.

Diagramas didácticos del mecanismo de atención y del pipeline de inferencia.

El autor pidió recursos visuales de alta calidad para repasar el proyecto y para usar en el artículo/presentaciones — no artifacts, archivos reales.

Se generaron dos figuras vectoriales (.svg/.pdf/.png) con matplotlib, construidas directamente a partir del código real (no una interpretación aproximada): `fig_mecanismo_atencion_mil` (el cálculo interno de `AttentionAggregator`: compuerta $V \odot U$, apilado de puntuaciones, softmax por fila, $z_k = \sum_i \alpha_{k,i} h_i$, con una skip connection dibujada explícitamente para h_i) y `fig_pipeline_foto_prediccion` (vista de alto nivel: foto $\rightarrow$ ROI recortado a mano $\rightarrow$ parches $\rightarrow$ red $\rightarrow$ probabilidades + heatmap, con la limitación del recorte manual señalada directamente en el diagrama). Ambas pasaron por varias rondas de retroalimentación visual real del autor (bloques con altura despareja, flechas cruzadas, texto desbordando su caja) hasta llegar a una versión con alturas de bloque uniformes y sin colisiones. La lección de proceso que vale la pena dejar anotada: el script final que genera estas figuras (carpeta `_proceso_claude/scripts/`) ya no ajusta posiciones a mano por prueba y error — deriva la altura de cada bloque sumando constantes con nombre, reparte el contenido de los bloques con menos material con una función `distribuir_verticamente()`, y verifica desbordes de texto con el *bounding box* realmente renderizado (`get_window_extent`) en vez de una estimación de caracteres — así, si el catálogo de clases crece o cambian los datos de ejemplo, la figura se puede regenerar sin repetir el mismo ciclo de ajuste manual.

Resultado real de la prueba a 112px. Continuación del punto sobre costo computacional de reducir el tamaño de parche.

Se entrenó (30 épocas, mismo split train/val/test que la corrida de 224px) y evaluó la red con parches de 112px. Loss final de entrenamiento: 0.9112 (vs. 0.8238 a 224px) — converge sano pero a un punto más alto, consistente con que a 112px cada bolsa tiene $\sim 4\times$ más parches ($N \approx 192$ vs. 48) y el softmax de atención debe repartirse entre muchas más opciones con el mismo presupuesto de épocas. Para que la comparación de AUC-ROC fuera realmente justa se reevaluó también el checkpoint de 224px sobre el split de prueba *actual* (46 bolsas, el manifiesto había crecido desde la última evaluación registrada): el resultado es mixto, con ventaja neta leve para 112px. Mejoran Fibrina (0.722 $\rightarrow$ 0.777, el tejido más difícil según el artículo italiano de referencia) y, marginalmente, Granulación e Hiperpigmentación; empeoran Epitelización (0.796 $\rightarrow$ 0.746) y Tejido Necrótico (0.698 $\rightarrow$ 0.668). Las métricas agregadas favorecen 112px: Hamming Loss 0.279 $\rightarrow$ 0.257, Exactitud de subconjunto 0.044 $\rightarrow$ 0.196, F1 ponderado 0.43 $\rightarrow$ 0.47. **No es una conclusión sólida todavía** — varias clases tienen 0–2 positivos en este split (su AUC se mueve ± 0.15 –0.30 con un solo caso) — pero es material legítimo para llevar a la reunión con el tutor en vez de decidir unilateralmente.

Verificación: el bug de catálogo fragmentado no se repitió, y se blindó contra el futuro. *El autor recordó que en una sesión anterior el sistema reportaba “pocas imágenes” para Piel Perilesional Sana y Tejido Necrótico cuando en realidad había más — un síntoma del bug de catálogo fragmentado resuelto en la Etapa 0 (21–22 de agosto)— y preguntó si había vuelto a pasar.*

Se auditaron las 204 bolsas de 224px y las 200 de 112px (función `etiquetas_no_reconocidas` de `catalogo_tejidos.py`): **0 etiquetas sin resolver en ambas carpetas.** El bug no se repitió; el fix de la Etapa 0 sigue vigente. Los conteos bajos del split de prueba resultaron ser

reales, no un síntoma de bug: Tejido Necrótico tiene 26 bolsas positivas en todo el dataset (solo le tocaron 5 en este split por variación normal del reparto) y Piel Perilesional Sana tiene apenas 7–10 (según la resolución) — es, genuinamente, el tejido más raro del catálogo. Para que esta pregunta no tenga que resolverse a mano cada vez, se dejaron dos piezas permanentes en el código (no un script de diagnóstico aparte): `catalogo_tejidos.py` ganó `auditar_etiquetas_no_reconocidas()` (escanea una carpeta de bolsas y devuelve, por archivo, qué etiquetas crudas no matchean el catálogo vigente) y `contar_positivos_por_clase()` (conteo real por clase en todo el dataset, no solo un split). Ambas funciones ahora corren automáticamente al inicio de `entrenar_comil.py` y de `evaluar_comil.py`: la primera imprime una alerta si encuentra etiquetas sin resolver, y la segunda hace que cada línea del reporte de AUC-ROC muestre el conteo del split *junto al* conteo del dataset completo (p. ej. `positivos=2/46 (dataset=7/200)`), así que la pregunta “¿es una clase rara de verdad o hubo mala suerte en el split?” se responde con solo leer el reporte, sin tener que auditar nada a mano.

Auditoría visual de los heatmaps: dos bugs y un problema metodológico de fondo.

El autor revisó personalmente los paneles generados del experimento 112px y reportó que le daban poca esperanza: sin referencia de qué significan los colores, con zonas sin cuadrricular, y con las zonas sí coloreadas cayendo sobre piel sana o fondo en vez de la úlcera.

Se inspeccionaron `UPD_001_bag.png` y `UPD_025__roi_01_bag.png` directamente. Confirmado: (1) la barra de color es invisible por un bug real en `visualizar_resultados.py` — se genera con `alpha=0`, transparencia que se contagia a la leyenda; (2) las zonas “sin cuadrricular” no son un error de dibujo sino el reflejo visual directo del hallazgo de atención plana del 23 de agosto: el código solo pinta un parche si su atención normalizada por bolsa supera 35%, y con atención casi uniforme la mayoría de los parches caen por debajo y se vuelven transparentes; (3) el hallazgo nuevo y más serio: con lesiones alargadas o diagonales, la caja rectangular que las contiene captura una fracción grande de piel sana de relleno, porque `generador_bolsas.py` solo permite anotar rectángulos alineados a los ejes. Achicar el parche ayuda a separar lesión de relleno *dentro* de una caja, pero si a la vez se usan ROIs más chicas y limpias para reducir ese relleno, se vuelve a caer en bolsas homogéneas sin heterogeneidad interna para que la atención discrimine — es la misma tensión de fondo (ver entrada siguiente) manifestándose de dos formas visuales distintas.

Tensión de protocolo de anotación (v1 vs. v2) y la pregunta de mejora sustancial. *Discusión extensa motivada por la entrada anterior: el autor preguntó explícitamente si sabemos ya cuál protocolo de anotación es mejor, y planteó la preocupación de fondo de si el proyecto va a lograr una mejora sustancial sobre su propio artículo previo (coautoría del autor, supervisión densa completamente supervisada) antes de someter formalmente la solicitud de tesis.*

Sobre el protocolo: no existe todavía una comparación controlada entre ROIs grandes/multi-tejido (“v1”, con margen deliberado de piel sana) y ROIs chicas/casi-un-solo-tejido (“v2”). Ambos optimizan objetivos parcialmente opuestos — v1 conserva heterogeneidad interna (mejor localización potencial, etiquetas más ambiguas), v2 da etiquetas limpias pero destruye esa heterogeneidad (el 97.8 % de bolsas homogéneas ya documentado). El autor priorizó explícitamente granularidad de localización sobre limpieza de clasificación, por ser la utili-

dad clínica real el objetivo del proyecto — esto inclina la decisión hacia v1, o mejor, hacia una arquitectura que no obligue a elegir entre ambos (ver estilo CAM más abajo). Se corrigió además un dato de capacidad: el techo real de imágenes disponibles no son las 100 de `Dataset_Experto_100`, son **268** en total (lote recibido el 6 de abril de 2026, documentado en `co_mil_bitacora.pdf` y `co_mil_bitacora_parte2.pdf`); las 168 restantes existen pero no están preparadas en el pipeline.

Sobre la mejora sustancial: la respuesta honesta es que **no se puede prometer que Co-MIL supere en Dice** al trabajo previo (ResUnet+Unet++, supervisión densa, 1000+ imágenes) — con 58–268 imágenes y etiquetas débiles, competir en la misma métrica contra supervisión densa con muchas más imágenes no es una expectativa realista, y no es un fracaso del proyecto sino una consecuencia aritmética de la información disponible. La mejora defendible está en ejes que el trabajo previo no abordaba: costo de anotación, aprendizaje continuo (Etapa 3, objetivo específico 3, sigue en 0 %), y viabilidad embebida. Si la localización sigue tan plana como hoy, esa narrativa alternativa tampoco se sostiene del todo, porque mostrarle al médico dónde está cada tejido requiere localización que funcione — de ahí la urgencia de la siguiente entrada.

Investigación dirigida (deep research) sobre alternativas al colapso de atención, verificada fuente por fuente. *A petición del autor se usó Gemini Deep Research (con un prompt extenso que incluía todo el contexto técnico y las restricciones reales del proyecto) para buscar alternativas más recientes al operador Sm (NeurIPS 2024) propuesto antes. Cada cita relevante del reporte se verificó de forma independiente contra la fuente primaria antes de aceptarla — el reporte contenía tanto hallazgos reales como detalles inventados.*

Verificado como real, cada uno contra su fuente primaria (arXiv/ResearchGate/PubMed): *AEM: Attention Entropy Maximization for Multiple Instance Learning based Whole Slide Image Classification* (Zhang, Y. et al., MICCAI 2025, arXiv:2406.15303) agrega un término de entropía a la función de pérdida para penalizar la concentración excesiva de atención, sin tocar la arquitectura; *ASMIL: Attention-Stabilized Multiple Instance Learning for Whole Slide Imaging* (Ye, L. et al., arXiv:2603.06658, marzo 2026) reemplaza softmax por una sigmoide normalizada para permitir que varios parches puntúen alto de forma independiente en vez de competir por una masa de probabilidad fija; *From Correlation to Causation: Max-Pooling-Based Multi-Instance Learning...* (Liu, X. et al., arXiv:2408.09449, método “FocusMIL”) diagnostica que la atención puede aprender “evidencia negativa” del fondo para clasificar bolsas positivas, el mismo mecanismo sospechado en cajas con mucho relleno de piel sana; *QG-MIL: A Gated Transformer Aggregator for Domain-Agnostic Multiple Instance Learning in Medical Imaging* (Zedda, L. et al., arXiv:2606.20027) — nota: el reporte de Gemini inventó el backrónimo “Qwen Gated”, el paper real solo dice “Gated”; *Geometric multi-instance learning for weakly supervised gastric cancer segmentation* (Huang, C., Xia, H., Xiao, X. et al., *npj Digital Medicine*, 2026) modela relaciones espaciales explícitas entre parches vía grafo, diagnosticando el mismo problema (“bolsa de parches desordenada que ignora topología”) casi con las mismas palabras que usamos aquí, pero de implementación bastante más cara (semanas, no 1 día). También se confirmó, con cifras exactas, un despliegue móvil real: ViT-Small (22M parámetros) empaquetado en una app iOS de 41MB con 50–80ms de inferencia, entrenado sobre DFUC2021 (dataset público, 5,955 imágenes) para clasificar isquemia/infección — preprint en Research Square, diciembre 2025, aún no revisado

por pares.

Advertencia que debe quedar explícita: **ninguno** de estos métodos (ni el operador Sm investigado antes) fue evaluado en fotos clínicas pequeñas parecidas a las nuestras — todos se probaron en laminillas de patología completas o tomografías, con miles de parches y cientos o miles de casos. Seguimos sin un precedente directo de este método funcionando a nuestra escala y modalidad. La sección de “opinión experta” del reporte de Gemini tenía además un tono de certeza absoluta impropio de investigación real (incluyendo cifras que no se pudieron verificar en la fuente) — se descartó como veredicto y se conserva solo lo verificado arriba.

Plan revisado para el 24 de agosto (reemplaza el prototipo de rediseño CAM-style que se había planeado para ese día): leer las ecuaciones exactas de AEM y ASMIL directo de los papers, implementar AEM primero (el cambio más barato: solo la función de pérdida, sin tocar arquitectura ni comprometer la compatibilidad con el clasificador ya entrenado), reentrenar y medir si el contraste de atención mejora de forma real sobre la mediana base de 0.04. Si el tiempo alcanza, probar también ASMIL. Se descarta el spot-check de SAM que se había planeado — la evidencia de hoy confirma que sin ajuste fino no da señal confiable en piel/heridas. Si el contraste mejora: evaluar si basta o si además hace falta el rediseño CAM-style. Si no mejora: pasar a planear ese rediseño y/o la supervisión mixta (MS-CLAM, ya documentada) como plan B con evidencia real detrás, no como salto al vacío.

Implementación real del rediseño estilo CAM y primer entrenamiento lanzado.

Tras el análisis de AEM/ASMIL, el autor pidió implementar directamente la vía con más sustento matemático real en vez de seguir explorando parches por regularización.

Antes de escribir el modelo completo se hizo una prueba de factibilidad barata (sin entrenar nada): tomar el mapa de features nativo de MobileNetV2 (congelado, solo pesos de ImageNet) sobre el ROI reconstruido de UPD_025__roi_01_bag (la lesión diagonal delgada) y visualizarlo. El resultado fue una franja de activación siguiendo visualmente la lesión real, con 1078 celdas nativas contra los 22 parches actuales — $49\times$ más fino, con 1 solo forward pass en vez de 22.

Con esa señal, se implementó la arquitectura real, sin reemplazar el pipeline original (ambos coexisten para poder comparar): CO-MIL/dataset_cam.py (CoMILDatasetCAM, reconstruye el ROI completo a partir de los parches ya guardados, reutilizando el mismo filtro de split y las mismas etiquetas que CoMILDataset), CO-MIL/Models/attention_mil_cam.py (InstanceEncoderCAM: un solo forward pass del ROI completo; AttentionAggregator y BagClassifier se importan de attention_mil.py sin ningún cambio, tal como se había previsto), y CO-MIL/entrenar_comil_cam.py (batch_size=1, porque cada ROI reconstruido tiene un tamaño físico distinto y no se puede empaquetar en un lote fijo como los parches uniformes; reutiliza calcular_pos_weights y congelar_backbone de entrenar_comil.py por import). Se verificó con una prueba de humo (forward + backward sobre 3 bolsas reales) antes de lanzar la corrida completa de 30 épocas en segundo plano la noche del 23 de agosto. Limitación conocida y documentada en el propio metadata.json del experimento: esta variante todavía no tiene aumento de datos, y el entrenamiento por bolsa individual (en vez de por lotes) hace la optimización más ruidosa — la comparación contra la corrida original no será perfectamente controlada, pero sí es la evidencia real que hacía falta.

Resultado real de la evaluación del rediseño CAM: mixto, no concluyente. *El*

entrenamiento terminó la misma noche; se evaluó de inmediato para no dejar la pregunta abierta.

`evaluar_comil.py` se adaptó para detectar la arquitectura del checkpoint (`checkpoint[.arquitectura]`) y ramificar internamente en vez de mantener un script aparte — un solo script sirve para ambas arquitecturas, a petición explícita del autor. Resultado sobre el split de prueba (46 bolsas): en clasificación, la mayoría de los AUC-ROC quedaron cerca o por debajo de 0.5 (p. ej. Granulación 0.415, Piel Perilesional Sana 0.136), claramente peor que la base por parches (0.7–0.9). En contraste de atención (47 pares bolsa-tejido positivos): mediana 0.022 (peor que la base de 0.04), pero promedio 0.082 ($2\times$ más alto) y máximo 0.38 (nunca antes visto con parches, el mejor previo era 0.27). **No es un éxito ni un fracaso limpio** — el patrón (máximo más alto, mediana peor, clasificación inestable) es consistente con las dos limitaciones ya conocidas de esta primera corrida: sin aumento de datos todavía en `CoMILDatasetCAM`, y `batch_size=1` (optimización más ruidosa que la corrida original). Puesto a elegir entre reintentar con aumento de datos o detenerse aquí por hoy, el autor eligió explícitamente detenerse — esta evaluación queda registrada como evidencia inconclusa de una sola corrida sin afinar, no como un veredicto sobre la arquitectura.

Con esto se da por cerrada la sesión de trabajo del 23 de agosto de 2026 — sesión larga, con avances reales pero también con una discusión metodológica de fondo que quedó documentada en detalle en `_proceso_claude/contexto/ROADMAP_PoC.md` y no debe repetirse desde cero la próxima vez. Pendiente inmediato de la siguiente sesión, si se retoma este hilo: agregar aumento de datos a `CoMILDatasetCAM` y reentrenar antes de sacar cualquier conclusión real sobre el rediseño CAM. Pendientes de decisión más amplios, antes de continuar con la Etapa 3 (aprendizaje continuo, que sigue en 0 % y no debería posponerse más): protocolo de anotación v1 vs. v2, estrategia de recorte de ROI en producción, y si el proyecto representa una mejora sustancial defendible sobre el trabajo previo del autor o si conviene reenmarcar la contribución.

1.5. 27 de agosto de 2026

Material visual de seguimiento para la reunión con el tutor. *Reunión de seguimiento programada para hoy con el Dr. José Eduardo Chairez Veloz, de carácter informal: no se buscaba presentar cada detalle técnico, sino un recurso visual que ayudara a tomar decisiones sobre el proyecto.*

Se preparó una presentación visual (publicada como página web, no como archivo `.pptx`) que resume en 11 diapositivas: el pipeline de inferencia y el mecanismo de atención (reutilizando `fig_pipeline_foto_prediccion` y `fig_mecanismo_atencion_mil`, ya documentadas); los cuatro problemas de higiene técnica corregidos desde la auditoría del 21 de agosto; la tabla de desempeño por tejido del primer entrenamiento válido (224px); el hallazgo de atención plana con la figura del límite del refinamiento DenseCRF; la comparación de los experimentos de 112px y del rediseño estilo CAM (incluida la imagen de la prueba de factibilidad, $49\times$ más resolución); un resumen de las alternativas de literatura investigadas (AEM, ASMIL, Focus-MIL); y, al final, las cinco decisiones abiertas que requieren la opinión del tutor (protocolo de anotación, arquitectura, encuadre de la contribución frente al artículo previo, aprendiza-

je continuo, y recorte de ROI en producción) junto con una recomendación preliminar de metodología de anotación para la sesión con el médico que se sumaría después de la reunión.

Composición espacial del catálogo de tejidos. *Pregunta directa del autor: ¿cuáles son los 10 tejidos más abundantes en espacio, no en número de bolsas, dentro del dataset anotado hasta ahora? Se agregó como diapositiva adicional a la presentación anterior.*

Se calculó, sobre las 204 bolsas de `Dataset_Experto_100` a 224px, el área acumulada de `bbox_expanded` (el recuadro ya expandido por `smart_expansion`, en píxeles cuadrados) de todas las bolsas donde cada tejido aparece como positivo, resolviendo las etiquetas crudas contra el catálogo vigente con las mismas funciones de `catalogo_tejidos.py` que usan `entrenar_comil.py` y `evaluar_comil.py` (0 etiquetas sin reconocer, consistente con las auditorías anteriores). Como las bolsas son multietiqueta, el área de una bolsa se suma para *cada* tejido presente en ella — esto mide la huella espacial de las anotaciones, no una segmentación pixel a pixel, que este dataset no tiene.

Cuadro 1.3: Top 10 tejidos por área espacial acumulada (`Dataset_Experto_100`, 224px, 204 bolsas).

#	Tejido	Área (Mpx ²)	Bolsas	Imágenes
1	Tejido de Granulación	16.36	58	31
2	Fibrina (Esfacelo)	14.45	41	21
3	Maceración / Eritema	12.83	28	15
4	Tejido Calloso (Hiperqueratosis)	8.13	12	7
5	Hipergranulación	6.62	8	8
6	Hiperpigmentación	5.83	8	4
7	Tejido Necrótico (Escala)	4.47	26	11
8	Piel Perilesional Sana	4.11	10	4
9	Epitelización	3.61	13	8
10	Exudado	3.21	8	6

(11. Tendón/Músculo Expuesto: 0.10 Mpx² en 1 bolsa; 12. Hueso Expuesto: 0 — sigue sin ningún ejemplo en todo el dataset.)

Dos lecturas quedaron registradas para la reunión: primero, área y frecuencia no siempre coinciden — Tejido Calloso aparece en solo 12 bolsas pero ya es el #4 en área (cajas individuales grandes), mientras que Maceración/Eritema está en 28 bolsas y es #3 en área total pero tenía solo 1 positivo en el split de prueba de 46 bolsas (tabla de la sección del 22 de agosto) — abundancia real en el dataset que el split de evaluación todavía no refleja. Segundo, esta métrica es sensible al protocolo de anotación (ver discusión de protocolo v1 vs. v2 en la sección del 23 de agosto): cajas con margen de piel sana inflan el área de todos los tejidos que coexisten en ellas, así que no mide el área real de cada tejido en la herida, solo cuánto espacio ocupan las anotaciones que lo incluyen.